Abdelwahid Benslimane

wahid.benslimane@gmail.com

Bidirectional LSTM for Human Action Recognition in UCF101 Videos

April 18, 2026.

Table of contents

1) Introduction p. 2

2) Dataset description p. 2

3) Technical approach

3.1) Frame sampling and extraction p. 3

3.2) Visual feature extraction p. 3

3.3) Dimensionality reduction p. 3

3.4) Temporal sequence modeling and classification p. 4

4) Code

4.1) Importing the Required Libraries p. 4

4.2) Data Integrity Check p. 5

4.3) Frame sampling and extraction p. 6

4.4) Removal of videos with fewer than 25 extracted frames p. 7

4.5) Feature extraction and creation of usable datasets p. 7

4.6) Dimensionality reduction with PCA p. 10

4.7) Training and evaluation of the Bi-LSTM p. 10

4.8) Quick analysis of the results p. 12

4.9) Training a Bi-LSTM on “raw” data (without PCA) p. 12

4.10) Training a Bi-LSTM on "raw" data with increased hidden_dim p. 13

5) Possible areas for improvement p. 13

1) Introduction

This document is intended as a report on a project consisting of building a computer vision model capable of recognizing the action taking place in a video from the UCF101 dataset. As this is a classification problem, the approach will be to use a supervised learning model that leverages visual features extracted from video frames using a CNN (ResNet-50), along with the temporal dimension of the video through a Bi-LSTM. The programming language used is Python, and the execution environment is Jupyter.

The dataset used is UCF101 itself, which contains 13,320 videos covering 101 human actions. It was created by the University of Central Florida and serves as a standard benchmark for evaluating computer vision models.

I chose this topic because of my particular interest in computer vision, and because I wanted to present in a didactic way the steps and the fundamental yet practical tools involved in carrying out a project in this field.

A French version of this report exists. It is in fact the French report that I wrote first, before translating it to make this work accessible to a wider audience. The comments and output strings in the code have been left in French, as they are few in number and quick to translate if needed. This in no way hinders the understanding of the algorithmic logic.

The notebook can be retrieved from this link:
https://github.com/abenslimaneakawahid/ml-projects/blob/main/Computer_Vision_UCF101.ipynb.

2) Dataset description

The UCF101 dataset is a widely used benchmark for human action recognition in videos. As mentioned in the introduction, it was created by researchers at the University of Central Florida and contains 13,320 video clips, primarily sourced from YouTube, distributed across 101 action classes. These actions span several domains, such as sports (Basketball, Surfing, etc.), everyday activities (Typing, Brushing Teeth, etc.), and human interactions (Handshake, Hugging, etc.).

According to information gathered from this UCF101 dataset presentation document, each video has a resolution of 320 × 240 pixels and a fixed frame rate of 25 frames per second (FPS). Based on the same document, clip duration ranges from 1.06 seconds (minimum) to 71.04 seconds (maximum), with an average duration of approximately 7.21 seconds. In total, the dataset represents over 27 hours of video footage.

The videos are organized into 25 groups per action, with each group containing 4 to 7 clips sharing similar characteristics (actors or background). This organization makes it possible to create standardized train/test splits, thereby facilitating performance comparison across different models.

The videos can be downloaded from this link. The dataset is structured as follows: a UCF101.zip archive contains a UCF-101 folder, which includes 101 subfolders, one per action class, each named after the corresponding action label. Each subfolder contains all the videos belonging to that class.

Each video filename follows this naming scheme: **v_{class_label}_{group_id}_{video_clip_id}**.

The fact that the class is clearly identifiable from the filename will be leveraged in the code to facilitate the creation of the data structures required for training and validating the model.

The split between training and test sets is based on files provided in the archive downloadable here. Specifically, I used the files trainlist01.txt and testlist01.txt. The first contains a list of paths to each video in the training set, while the second contains a list of paths to the videos in the test set for the first split (three splits are provided in total). A second column contains the numeric identifier of the class each video belongs to, but this will not be used. Together, these two files cover the entire dataset (a point verified in the code).

3) Technical approach

3.1) Frame sampling and extraction

The main challenge is to capture both the spatial structure of each frame (objects, visual context) and the temporal dynamics of the sequence. A video is first broken down into individual frames, and since video duration varies, it is necessary to normalize sequence length. I chose to sample 25 frames per video, evenly distributed, after taking into account the frame rate, the average clip duration (7.21 seconds), and the distribution of durations per class.

Since most clips last between 2 and 10 seconds, sampling 25 frames amounts to retaining approximately 1 frame out of every 7 on average (around 180 frames per video at 25 FPS). For shorter clips (2 s / 50 frames) or longer ones (10 s / 250 frames), this ratio remains close to 1 in 2 and 1 in 10 respectively, ensuring sufficient temporal coverage.

Having a fixed number of frames per video also simplifies training the Bi-LSTM, which expects sequences of identical length. An alternative would have been to extract a variable number of frames and use padding and masking, but the moderate variability in clip durations does not justify this added complexity.

3.2) Visual feature extraction

Still in the spirit of finding the right trade-off, for the next step, which consists of extracting visual features from each frame, ResNet-50 (26M parameters, 4 GFLOPs/image, according to information available on this page) was selected for its good balance between efficiency and feature extraction capability. By comparison, ResNet-101 and ResNet-152 require approximately 2x and 3x more computation respectively, for an uncertain gain in the context of this project.

Before being passed through the CNN, each frame is resized to the format (3, 224, 224), then normalized channel by channel (RGB, the "3" in the first dimension) using the ImageNet mean and standard deviation, as the model was pre-trained on these values. The final fully connected layer is removed in order to produce, for each frame, a 2048-dimensional vector derived from global average pooling. For a complete video, this yields a tensor of dimensions (25, 2048) at the output of the CNN.

3.3) Dimensionality reduction

Since the frame feature vectors are high-dimensional, I sought to compress their format while preserving the essential information they carry. To do so, I applied Principal Component Analysis (PCA) and retained the first 256 dimensions. This approach makes it possible to keep the bulk of the discriminative information (as will be demonstrated later in the code) while eliminating redundant or noisy information. This offers two concrete benefits: reducing the computational cost of the Bi-LSTM, and limiting the risk of overfitting by avoiding feeding it uninformative dimensions.

However, for comparison purposes and in the interest of finding a better solution, a model will also be trained on the raw CNN output data, without PCA.

3.4) Temporal sequence modeling and classification

The sequences of 25 feature vectors each will then be processed in both directions (past -> future and future -> past) by a Bi-LSTM, enabling the model to capture temporal dependencies in both directions. The hidden outputs from both directions are concatenated at each time step, and it is the output corresponding to the last time step that is retained and subsequently used for classification.

The vector produced at the last time step is passed through a fully connected layer that produces one score per class. The predicted class is the one with the highest score. The model is trained with a CrossEntropyLoss and the Adam optimizer over 15 epochs.

The fully connected layer produces a vector of 101 raw scores called logits (one score per class). These logits are then converted into probabilities by a softmax layer. The index of the highest score, corresponding to the predicted class, can then easily be decoded into a class name to retrieve the human-readable action label.

Technical details have been intentionally omitted from this section and will be elaborated upon in the part dedicated to the code.

The overall technical approach can be summarized in the simplified flow diagram below:

Sampling of 25 frames ▶️ Visual feature extraction via ResNet-50 ▶️ Temporal dependency modeling with a Bi-LSTM ▶️Temporal dependency modeling with a Bi-LSTM ▶️ Classification using a fully connected layer + Softmax ▶️ Class prediction.

.4) Code

4.1) Importing the Required Libraries

In the first code block, I gathered all the import statements for the various libraries used throughout the project, so as to resolve from the outset any technical issues related to this aspect, in case the code were to be run in a different environment.

```
import cv2
import math
import matplotlib.pyplot as plt
%matplotlib inline
import pandas as pd
from keras.preprocessing import image
import numpy as np
from tensorflow.keras.utils import to_categorical
from skimage.transform import resize
from sklearn.model_selection import train_test_split
from glob import glob
from tqdm import tqdm
import os
from pathlib import Path
import shutil
import os
from sklearn.decomposition import PCA
from sklearn.preprocessing import LabelEncoder
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, TensorDataset
from torchvision import models, transforms
from PIL import Image
import joblib
from sklearn.metrics import classification_report
```

4.2) Data Integrity Check

The **load_file** function iterates over the split files, trainlist01.txt and testlist01.txt, extracting only the video paths (the label is ignored). The duplicates function then verifies the integrity of the splits in two steps: detection of internal duplicates via **set()**, followed by detection of data shared between the two files via **intersection()**, ensuring a strict separation of the two datasets before proceeding further. The results show that integrity is guaranteed, in other words, trainlist01.txt contains 9,537 videos, testlist01.txt contains 3,783 videos (total: 13,320), with no duplicates, and no data common to both.

```python
def load_file(path):
    videos = []
    with open(path, "r") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            video_path = line.split()[0]  # on ignore le label
            videos.append(video_path)
    return videos


files = [
    "trainlist01.txt",
    "testlist01.txt"
]


def duplicates(files):
    file_videos = {}

    # Vérifier doublons internes

    for file in files:
        videos = load_file(file)
        unique_videos = set(videos)

        print(f"\nFichier : {file}")
        print("Total lignes :", len(videos))
        print("Unique :", len(unique_videos))

        if len(videos) != len(unique_videos):
            print("Doublons détectés dans le fichier !")
        else:
            print("Aucun doublon interne")

        file_videos[file] = unique_videos


    # Vérifier intersections entre fichiers

    for i in range(len(files)):
        for j in range(i + 1, len(files)):
            f1 = files[i]
            f2 = files[j]

            intersection = file_videos[f1].intersection(file_videos[f2])

            print(f"\nIntersection {f1} / {f2} : {len(intersection)}")

            if len(intersection) > 0:
                print("Vidéos communes détectées !")
            else:
                print("Aucune vidéo commune")
                print(f"\nTotal vidéos:  {len(file_videos[f1]) + len(file_videos[f2])}")

duplicates(files)
```

4.3) Frame sampling and extraction

The lists of paths to the train and test videos are loaded via **load_file**. The **extract_frames** function then iterates over each video: it opens the file with **cv2.VideoCapture**, selects 25 evenly distributed frames across the total duration via **np.linspace**, and saves each frame as a .jpg file in a dedicated folder per video. Uniform sampling ensures consistent temporal coverage regardless of video duration.

```
trainfile = load_file("trainlist01.txt")
testfile = load_file("testlist01.txt")

#extraction des frames

def extract_frames(video_list,
                   videos_root,
                   output_root,
                   nb_frames=25):

    os.makedirs(output_root, exist_ok=True)

    for video in video_list:

        video_path = videos_root+"/"+video

        if not os.path.exists(video_path):
            print(f"Fichier introuvable : {video_path}")
            continue

        video_name = os.path.splitext(os.path.basename(video_path))[0]

        # 1 dossier par vidéo
        video_dir = os.path.join(output_root, video_name)
        os.makedirs(video_dir, exist_ok=True)

        cap = cv2.VideoCapture(video_path)

        if not cap.isOpened():
            print(f"Impossible d'ouvrir : {video_path}")
            continue

        total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

        if total_frames <= 0:
            cap.release()
            continue

        frame_indices = np.linspace(0, total_frames - 1, nb_frames, dtype=int)

        for i, idx in enumerate(frame_indices):
            cap.set(cv2.CAP_PROP_POS_FRAMES, int(idx))
            ret, frame = cap.read()

            if ret:
                frame_filename = f"{video_name}_frame_{i}.jpg"
                frame_path = os.path.join(video_dir, frame_filename)
                cv2.imwrite(frame_path, frame)

        cap.release()

    print("Extraction terminée.")


extract_frames(trainfile, "UCF-101", "frames_training", nb_frames=25)
extract_frames(testfile, "UCF-101", "frames_test", nb_frames=25)
```

4.4) Removal of videos with fewer than 25 extracted frames

The **nb_frames_check** function iterates over each video folder and detects those not containing exactly 25 frames, a situation that can arise with videos that are too short or corrupted, after which **delete_wrong_folders** removes these folders via **shutil.rmtree**. This step ensures that all samples have exactly the same sequence length. Since the number of detected videos is small (42 in the training set, 13 in the test set), and given that no affected class is significantly impacted, this removal will have no effect on training (no bias to be concerned about).

```
#vérification du nombre de frames extraites à partir de chaque vidéo

def nb_frames_check(path, nb):

    parent_dir = Path(path)

    wrong_folders = []

    for folder in parent_dir.iterdir():
        if folder.is_dir():
            files = [f for f in folder.iterdir() if f.is_file()]
            if len(files) != nb:
                wrong_folders.append(folder.name)
    print(f"nombre de dossiers ne contenant pas {nb} frames :")
    print(len(wrong_folders))
    print("\n")
    if len(wrong_folders) !=0:
        print(f"dossiers ne contenant pas {nb} frames :")
        print(wrong_folders)

    return wrong_folders

wrong_folders_train = nb_frames_check("frames_training", 25)
print("\n")
wrong_folders_test = nb_frames_check("frames_test", 25)

#suppression des dossiers contenant moins de 25 frames


def delete_wrong_folders(parent_dir_str, wrong_folders):
    parent_dir = Path(parent_dir_str)


    for folder_name in wrong_folders:
        folder_path = parent_dir / folder_name
        if folder_path.exists() and folder_path.is_dir():
            shutil.rmtree(folder_path)

    print("Suppression terminée.")

delete_wrong_folders("frames_training", wrong_folders_train)
delete_wrong_folders("frames_test", wrong_folders_test)
```

4.5) Feature extraction and creation of usable datasets

This section creates a **VideoDataset** for training and one for validation, loading for each video its 25 frames in chronological order (guaranteed by the sort ***sorted(frames, key=lambda x: int(x.split("_")[-1].split(".")[0]))*** which orders frames by their final numeric index). Each frame is resized to 224 x 224, converted to a tensor, and normalized before being stacked into a tensor of dimensions (25, 3, 224, 224).

The visual features are then extracted by a pre-trained ResNet-50 (without its final layer), producing for each video a tensor of dimensions (25, 2048), which means a 2048-dimensional feature vector per frame. The entire training set is thus represented by an array of dimensions (number of videos, 25, 2048).

Finally, the training set videos are randomly shuffled via **np.random.permutation** (with a seed fixed at 42 for reproducibility). This shuffling is necessary because, without it, batches could contain consecutive videos of the same class and bias the learning process. The chronological order of frames within each video is, however, preserved, as the Bi-LSTM depends on it to model the temporal sequence.

```
#création du Dataset contenant les liens vers les fichiers vidéos et les labels correspondants
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")


class VideoDataset(Dataset):
    def __init__(self, frames_dir, transform=None):
        self.samples = []
        self.transform = transform
        self.labels = []

        for video_folder in os.listdir(frames_dir):
            video_path = os.path.join(frames_dir, video_folder)
            if not os.path.isdir(video_path):
                continue

            # Extraire la classe depuis le nom du dossier
            cls = video_folder.split('_')[1]
            self.labels.append(cls)

            # Lister les frames
            #frames = sorted([os.path.join(video_path, f)
            #                 for f in os.listdir(video_path) if f.endswith(".jpg")])
            frames = [os.path.join(video_path, f)
                      for f in os.listdir(video_path)]
            frames = sorted(frames, key=lambda x: int(x.split("_")[-1].split(".")[0]))

            self.samples.append(frames)

        self.label_encoder = LabelEncoder()
        self.label_encoder.fit(self.labels)
        self.labels = self.label_encoder.transform(self.labels)

    def __len__(self):
        return len(self.samples)

    #retourne les 25 frames pour chaque vidéo ainsi que le label de la classe
    def __getitem__(self, idx):
        frames_paths = self.samples[idx]
        frames_tensor = []

        for fpath in frames_paths:
            img = Image.open(fpath).convert("RGB")
            if self.transform:
                img = self.transform(img)
            frames_tensor.append(img)

        frames_tensor = torch.stack(frames_tensor) #dimensions: (nb frames, 3, 224, 224)
        label = self.labels[idx]
        return frames_tensor, label

#normalisation des frames en vue de l'extraction de feature avec Resnet-50:
https://docs.pytorch.org/vision/0.9/models
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
```

```
                        std=[0.229, 0.224, 0.225])
])

dataset_train = VideoDataset("frames_training", transform=transform)
dataloader_train = DataLoader(dataset_train, batch_size=1, shuffle=False)
dataset_test = VideoDataset("frames_test", transform=transform)
dataloader_test = DataLoader(dataset_test, batch_size=1, shuffle=False)

def extraction_features(dataloader):

    cnn_model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
    cnn_model = nn.Sequential(*list(cnn_model.children())[:-1])  # retire la dernière couche FC
    cnn_model.eval()
    cnn_model.to(DEVICE)

    # Extraction des features
    all_features = []
    all_labels = []

    with torch.no_grad():
        for frames, label in tqdm(dataloader, desc="Extraction features"):
            frames = frames.squeeze(0).to(DEVICE)  # dimensions: (nb frames, 3, 224, 224)
            features = cnn_model(frames)            # (nb frames, 2048, 1, 1)
            features = features.squeeze(-1).squeeze(-1)  # (nb frames, 2048)
            all_features.append(features.cpu().numpy())
            all_labels.append(label.item())

    all_features = np.array(all_features)  # dimensions: (nb videos, nb frames, 2048)
    all_labels = np.array(all_labels)

    return all_features, all_labels

dataset_train = VideoDataset("frames_training", transform=transform)
dataloader_train = DataLoader(dataset_train, batch_size=1, shuffle=False)
dataset_test = VideoDataset("frames_test", transform=transform)
dataloader_test = DataLoader(dataset_test, batch_size=1, shuffle=False)

#Extraction des features visuelles avec ResNet-50

def extraction_features(dataloader):

    cnn_model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
    cnn_model = nn.Sequential(*list(cnn_model.children())[:-1])  # retirer la dernière couche FC
    cnn_model.eval()
    cnn_model.to(DEVICE)

    # Extraire les features
    all_features = []
    all_labels = []

    with torch.no_grad():
        for frames, label in tqdm(dataloader, desc="Extraction features"):
            frames = frames.squeeze(0).to(DEVICE)  # dimensions: (nb frames, 3, 224, 224)
            features = cnn_model(frames)            # (nb frames, 2048, 1, 1)
            features = features.squeeze(-1).squeeze(-1)  # (nb frames, 2048)
            all_features.append(features.cpu().numpy())
            all_labels.append(label.item())

    all_features = np.array(all_features)  # dimensions: (nb videos, nb frames, 2048)
    all_labels = np.array(all_labels)

    return all_features, all_labels

all_features_train, all_labels_train = extraction_features(dataloader_train)
all_features_test, all_labels_test = extraction_features(dataloader_test)

#on mélange les vidéos de manière à ne pas à s'assurer que les classes sont réparties de manière totalement
aléatoires
np.random.seed(42)
indices = np.random.permutation(len(all_features_train))
all_features_train = all_features_train[indices]
all_labels_train = all_labels_train[indices]
```

4.6) Dimensionality reduction with PCA

The data of dimensions (nb videos, 25, 2048) is flattened into a new format (nb videos x 25, 2048) to be compatible with PCA, which reduces the dimensionality from 2048 to 256 components, before being reshaped back to (nb videos, 25, 256). The retained explained variance ratio is computed and displayed, amounting to **93.75%**.

```
#ACP

n_videos, n_frames, n_features = all_features_train.shape
X_train_flat = all_features_train.reshape(n_videos*n_frames, n_features)
pca = PCA(n_components=256)

X_train_pca = pca.fit_transform(X_train_flat)
X_train_pca = X_train_pca.reshape(n_videos, n_frames, 256)

inertie = np.sum(pca.explained_variance_ratio_) * 100
print(f"Part d'inertie retenue : {inertie:.2f}%")

#on enregistre l'espace réduit produit
#joblib.dump(pca, "pca_features.pkl")
```

4.7) Training and evaluation of the Bi-LSTM

In this section, I create a Bi-LSTM with the following parameters: input_dim=256 (dimensionality from PCA), hidden_dim=128 (256 after concatenation of both directions), a 30% dropout applied to the output of the last time step, and a fully connected layer 256 -> 101 for classification. The PCA-transformed training data is wrapped in a TensorDataset and loaded in batches of 32 samples with shuffling, ensuring videos are mixed at each epoch (in addition to the shuffling applied to the data prior to PCA). The model is trained with a CrossEntropyLoss and the Adam optimizer over 15 epochs.

For evaluation, the test data is first projected into the same PCA space computed previously, then passed all at once to the model in **eval()** mode, which disables dropout. The predicted class is determined by **torch.max** on the logits. The model is then saved via **state_dict()** so that it can be reloaded later if needed, without retraining.

The accuracy achieved on the test data amounts to **75.20%**. This result will be analyzed in slightly more detail below.

```
# LSTM bidirectionnel pour classification avec dropout de 30%

torch.manual_seed(42)
class BiLSTMClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_classes, dropout):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True, bidirectional=True)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim*2, num_classes)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = out[:, -1, :]        # on garde la sortie du dernier timestep
        out = self.dropout(out)
        out = self.fc(out)
        return out

model = BiLSTMClassifier(input_dim=256, hidden_dim=128, num_classes=101, dropout=0.3).to(DEVICE)

#création des tenseurs de données issues de l'ACP
```

```python
features_tensor = torch.tensor(X_train_pca, dtype=torch.float32)
labels_tensor = torch.tensor(all_labels_train, dtype=torch.long)

dataset_lstm = TensorDataset(features_tensor, labels_tensor)
dataloader_lstm = DataLoader(dataset_lstm, batch_size=32, shuffle=True)


# Entraînement du modèle bi-LSTM + couche entièrement connectée

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
epochs = 15

def training(model, dataloader, optimizer, criterion, epochs):

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for batch, (batch_features, batch_labels) in enumerate(dataloader):

            # envoyer sur device
            batch_features = batch_features.to(DEVICE)
            batch_labels = batch_labels.to(DEVICE)

            optimizer.zero_grad()

            outputs = model(batch_features)
            loss = criterion(outputs, batch_labels)

            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        print(f"Epoch {epoch + 1}/{epochs} - Average Loss: {total_loss/len(dataloader):.4f}")

    print("Entraînement terminé !")

training(model, dataloader_lstm, optimizer, criterion, epochs=15)

#évaluation du modèle
n_videos_test, n_frames, n_features = all_features_test.shape
X_test_flat = all_features_test.reshape(n_videos_test * n_frames, n_features)
X_test_pca = pca.transform(X_test_flat)
all_features_test_pca = X_test_pca.reshape(n_videos_test, n_frames, 256)

model.eval()

features_test_tensor = torch.tensor(all_features_test_pca, dtype=torch.float32).to(DEVICE)
labels_test_tensor = torch.tensor(all_labels_test, dtype=torch.long).to(DEVICE)

with torch.no_grad():
    outputs_test = model(features_test_tensor)
    _, preds_test = torch.max(outputs_test, 1)
    accuracy_test = (preds_test == labels_test_tensor).float().mean()

print(f"Accuracy test : {accuracy_test.item():.4f}")

#enregistrement sur disque du modèle

torch.save(model.state_dict(), "bilstm_inputdim256.pth")

#pour recharger:
#model = BiLSTMClassifier(input_dim=256, hidden_dim=128, num_classes=101, dropout=0.3).to(DEVICE)
#model.load_state_dict(torch.load("bilstm_model.pth", map_location=DEVICE))
```

4.8) Quick analysis of the results

In this section, I use the **classification_report** function, which computes precision, recall, and f1-score for each of the 101 classes, replacing numeric indices with human-readable action names via **dataset_train.label_encoder.classes_**. It also displays the overall averages: macro avg (unweighted average per class) and weighted avg (weighted by the number of examples per class), as well as the overall accuracy. This allows me to make a few observations.

I notice that among the videos with a high f1-score (> 0.80) are those involving the playing of a musical instrument (Drumming, PlayingGuitar, PlayingSitar, PlayingFlute, etc.). One can assume that the presence of the musical instrument serves as a strong visual signal that allows the model to recognize the action easily, especially if the instrument is visible from the very first frames. At the same time, these classes, according to Figure 3 of the UCF101 dataset presentation document (see the **Dataset description** section of this report), tend to have longer videos (> 5 s), for which 25 frames do not densely cover the action. A strong visual signature therefore compensates for a less thorough coverage of the temporal stream by the model.

More generally, activities, particularly sports, that take place in a distinctive, specific, and clearly identifiable environment, and that can be recognized from a single frame or very few frames, tend to achieve high f1-scores. Examples include RockClimbingIndoor, HorseRace, HorseRiding, IceDancing, etc.

Conversely, activities for which the overall visual context does not necessarily provide relevant information, or which do not inherently involve distinctive objects, obtain low f1-scores: JumpRope (0.00), BodyWeightSquats (0.17), etc. Indeed, these activities can take place in very different contexts, as can be observed by watching a few videos from the JumpRope class, for example. Static visual signature therefore appears to be an important discriminating factor for the model.

This analysis is of course brief and remains at a surface level. A more in-depth study taking into account a cross-analysis of the various aspects characterizing the videos would be necessary to compare results and identify patterns leading to poor or strong classification performance in most cases.

```
#analyse détaillée des résultats

preds_np = preds_test.cpu().numpy()
labels_np = labels_test_tensor.cpu().numpy()

print(classification_report(
    labels_np,
    preds_np,
    target_names=dataset_train.label_encoder.classes_
))
```

4.9) Training a Bi-LSTM on "raw" data (without PCA)

A Bi-LSTM is trained this time without PCA, using the "raw" features from ResNet-50 as input (input_dim=2048 instead of 256), with all other hyperparameters remaining identical (hidden_dim=128, dropout=0.3, 15 epochs, batch_size=32). The accuracy obtained is **72.39%**, slightly lower than that achieved with PCA, suggesting that in this case dimensionality reduction did not degrade the discriminative information and in fact helped the model generalize better, while also enabling faster training and convergence at a lower computational cost.

```python
#entraînement d'un modèle avec en entrée des données de dimension (25, 2048)

model = BiLSTMClassifier(input_dim=2048, hidden_dim=128, num_classes=101, dropout=0.3).to(DEVICE)

features_tensor = torch.tensor(all_features_train, dtype=torch.float32)
labels_tensor = torch.tensor(all_labels_train, dtype=torch.long)
features_tensor = torch.tensor(all_features_train, dtype=torch.float32)
labels_tensor = torch.tensor(all_labels_train, dtype=torch.long)

dataset_lstm = TensorDataset(features_tensor, labels_tensor)
dataloader_lstm = DataLoader(dataset_lstm, batch_size=32, shuffle=True)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

training(model, dataloader_lstm, optimizer, criterion, epochs=15)

#évaluation du nouveau modèle précédent

model.eval()

features_test_tensor = torch.tensor(all_features_test, dtype=torch.float32).to(DEVICE)
labels_test_tensor = torch.tensor(all_labels_test, dtype=torch.long).to(DEVICE)

with torch.no_grad():
    outputs_test = model(features_test_tensor)
    _, preds_test = torch.max(outputs_test, 1)
    accuracy_test = (preds_test == labels_test_tensor).float().mean()

print(f"Accuracy test : {accuracy_test.item():.4f}")
```

4.10) Training a Bi-LSTM on "raw" data with increased hidden_dim

This time, a model with hidden_dim doubled to 256 (512 after concatenation of both directions) is trained again on raw ResNet-50 features to increase the model's capacity, with all other parameters remaining identical. The accuracy drops to **69.84%**, indicating that the increased capacity has reduced generalization ability. With only 15 epochs and no data augmentation, the model is unable to make good use of the additional parameters.

```python
#maintenant, augmentons le paramètre hidden_dim pour voir si les résultats sont meilleurs

model = BiLSTMClassifier(input_dim=2048, hidden_dim=256, num_classes=101, dropout=0.3).to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

training(model, dataloader_lstm, optimizer, criterion, epochs=15)

with torch.no_grad():
    outputs_test = model(features_test_tensor)
    _, preds_test = torch.max(outputs_test, 1)
    accuracy_test = (preds_test == labels_test_tensor).float().mean()

print(f"Accuracy test : {accuracy_test.item():.4f}")
```

5) Possible areas for improvement

Several avenues can be considered to attempt to improve the model's performance. The first would be to fine-tune ResNet-50 on UCF101 rather than using its frozen weights. The extracted visual features could then be better suited to recognizing certain actions. Along the same lines, one could also replace ResNet-50 with a more recent ResNet or CNN (such as EfficientNet or ConvNeXt) and

fine-tune it or not on the dataset frames. Finally, another interesting avenue would be to train several specialized models on subsets of classes grouped by clip duration, having one model dedicated to short clips and another to long clips, in order to adapt the sequence length processed by the Bi-LSTM to the duration of the videos, which would mitigate potential issues arising from uniform sampling at 25 frames.

One can also act on the (vertical) depth of the Bi-LSTM by adding a layer, or opt for the use of a Transformer. A combination of all these approaches would likely yield better results, but this implies increasing the volume of data, particularly if one wishes to have models dedicated to short and long clips respectively.

For comparison purposes, the following study is worth mentioning: Video-Based Human Activity Recognition Using Hybrid Deep Learning Model. It combines a pre-trained ConvNeXt CNN with a Vision Transformer for feature extraction, followed by a Bi-LSTM or TCN for temporal modeling, and achieves 98.46% accuracy on UCF101.